

دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش آزمایش ۸

ALU اعداد مختلط

Digital Systems Design Laboratory

اعضای گروه

پوریا عرفانزاده ۴۰۱۱۱۰۹۱۸
فاطیما تیمارچی ۴۰۲۱۰۵۸۰۲
آیه صابری ۴۰۲۱۰۶۱۴۵

نام درس

آزمایشگاه طراحی سیستم‌های دیجیتال

استاد درس

دکتر علیرضا اجلالی

تابستان ۱۴۰۵

۱.۸ مقدمه و اهداف

این آزمایش از چند بخش تشکیل شده است. مطابق دستورکار، ابتدا باید پیمانه‌های گوناگون پیاده‌سازی و آزموده شوند و سپس با کنار هم گذاشتن آن‌ها یک رایانهٔ پایه ساخته شود که حافظه‌ای ۳۲ کلمه‌ای دارد. سه خواستهٔ صریح دستورکار عبارت‌اند از پیمانهٔ جمع و تفریق اعداد مختلط، پیمانهٔ ضرب اعداد مختلط، و واحدی که دستورها را از همان حافظه بخواند و عملیات مختلط را اجرا کند.

افزون بر این‌ها، دستورکار یک قید مهم گذاشته است: «نبایستی از واحدهای جمع‌کننده یا ضرب‌کننده بیش از یک بار استفاده شود». این قید، شکل کلّی معماری طرح را تعیین می‌کند و در سراسر این گزارش به آن بازخواهیم گشت؛ چراکه رعایت آن به‌معنای پرهیز از هرگونه موازی‌سازی حسابی و تکیه بر اشتراک زمانی[۱] منابع است.

روند انجام آزمایش

نخست دو بلوک حسابی مشترک، یعنی یک جمع‌کننده/تفریق‌کننده و یک ضرب‌کنندهٔ علامت‌دار، به‌صورت پیمانه‌های ترکیبی کوچک نوشته شدند. سپس چهار پیمانهٔ عملیاتی — جمع و تفریق، ضرب، مزدوج و مربع اندازه — هر یک به‌صورت یک ماشین حالت چندچرخه‌ای[۲] ساخته شدند که در هر چرخه تنها یک بار از بلوک حسابی مشترک استفاده می‌کند. برای هر پیمانه یک میز آزمون[۳] خودآزما نوشته و جداگانه اجرا شد. در گام بعد، همهٔ این عملیات در یک واحد حساب و منطق مختلط یکپارچه شدند و سرانجام این واحد به یک حافظهٔ دستور ۳۲ کلمه‌ای و یک واحد کنترل متّصل شد تا رایانهٔ خواسته‌شده کامل گردد. پس از تأیید همهٔ آزمون‌ها در ModelSim، طرح در Quartus II سنتز و تحلیل زمانی شد.

۲.۸ نمایش اعداد و ساختار داده

الف) قالب عدد مختلط

هر عدد مختلط با دو بخش حقیقی و موهومی نمایش داده می‌شود و هر بخش یک عدد علامت‌دار هشت‌بیتی در نمایش متمّم دو[۴] است؛ یعنی هر بخش در بازهٔ -128 تا $+127$ قرار می‌گیرد. پهنای ورودی با پارامتر WIDTH بیان شده و همهٔ پیمانه‌ها آن را می‌پذیرند، بنابراین طرح به عدد هشت وابستگی سخت ندارد.

[1] Time-Multiplexed Resource Sharing
[2] Multi-Cycle Design
[3] Testbench
[4] Two's Complement

پهنای خروجی از رابطهٔ $\text{RESULT_WIDTH} = 2 \times \text{WIDTH} + 1$ به دست می‌آید که برای ورودی هشت‌بیتی برابر هفده بیت است. دلیل این انتخاب در ضرب مختلط نهفته است: حاصل‌ضرب دو عدد هشت‌بیتی علامت‌دار شانزده بیت می‌شود و بخش حقیقی نتیجه از تفریق دو چنین حاصل‌ضربی به دست می‌آید، پس برای جلوگیری از سرریز[5] به یک بیت اضافی نیاز است. با هفده بیت، هیچ‌یک از چهار عملیات طرح نمی‌تواند از پهنای خروجی بیرون بزند.

ب) قالب دستور و حافظه

حافظهٔ دستور همان ۳۲ کلمهٔ خواسته‌شدهٔ دستورکار است. هر کلمه سی و پنج بیت دارد و از الحاق یک کد عملیات سه‌بیتی با چهار میدان هشت‌بیتی ساخته می‌شود:

$$\underbrace{\text{opcode}}_{3},\ \underbrace{a_{\text{re}}}_{8},\ \underbrace{a_{\text{im}}}_{8},\ \underbrace{b_{\text{re}}}_{8},\ \underbrace{b_{\text{im}}}_{8}$$

یعنی هر دستور، هم عملیات و هم هر دو عملوند مختلط خود را در دل خود دارد. این انتخاب، طرح را از داشتن فایل ثبات یا حافظهٔ داده بی‌نیاز می‌کند و تمرکز آزمایش را روی خود واحد حساب و اشتراک منابع نگه می‌دارد.

کد	نام	عملیات
000	CADD	جمع دو عدد مختلط
001	CSUB	تفریق دو عدد مختلط
010	CMUL	ضرب دو عدد مختلط
011	CONJ	مزدوج عملوند نخست
100	MAG2	مربع اندازه‌ی عملوند نخست
101	NOP	بدون عملیات

جدول ۱: مجموعه‌ی دستورهای رایانه‌ی اعداد مختلط

۳.۸ منابع حسابی مشترک

کلّ سخت‌افزار حسابی طرح تنها از دو بلوک ترکیبی کوچک تشکیل شده است: یک جمع‌کننده/تفریق‌کننده که با یک بیت انتخاب میان جمع و تفریق سوئیچ می‌کند، و یک ضرب‌کنندهٔ علامت‌دار. همهٔ عملیات مختلط طرح، بدون استثنا، از همین دو بلوک استفاده می‌کنند.

[5] Overflow

```
// Small combinational arithmetic blocks used as the shared hardware resources in the multi-cycle
    design.
module shared_addsub #(
    parameter WIDTH = 17
) (
    input  signed [WIDTH-1:0] a,
    input  signed [WIDTH-1:0] b,
    input                     sub,   // 0: a+b, 1: a-b
    output signed [WIDTH-1:0] y
);
    assign y = sub ? (a - b) : (a + b);
endmodule

// Signed WIDTH x WIDTH multiplier. Only one instance is used in the main complex ALU.
module shared_multiplier #(
    parameter WIDTH = 8
) (
    input  signed [WIDTH-1:0]     a,
    input  signed [WIDTH-1:0]     b,
    output signed [2*WIDTH-1:0]   y
);
    assign y = a * b;
endmodule
```

کد ۱: دو بلوک حسابی مشترک

اهمّیّت این ساختار در برابر قید دستورکار روشن است. اگر ضرب مختلط به شیوهٔ متعارف و موازی نوشته می‌شد، به چهار ضرب‌کننده و دو جمع‌کننده نیاز داشت، چون

$$(a + jb)(c + jd) = (ac - bd) + j(ad + bc).$$

در این طرح به‌جای آن، همان یک ضرب‌کننده در چهار چرخهٔ پیاپی برای ac، bd، ad و bc به کار می‌رود و سپس همان یک جمع‌کننده در دو چرخهٔ دیگر تفریق و جمع پایانی را انجام می‌دهد. بهای این کار، طولانی‌تر شدن زمان اجرای هر عملیات است؛ سودش، رعایت قید دستورکار و کاهش چشمگیر مساحت.

۴.۸ پیمانه‌های عملیاتی

الف) جمع و تفریق مختلط

این پیمانه چهار حالت دارد: بی‌کار، محاسبهٔ بخش حقیقی، محاسبهٔ بخش موهومی، و پایان. در حالت بی‌کار هر دو عملوند در ثبات‌های درونی نگه داشته می‌شوند؛ سپس در دو چرخهٔ پیاپی، همان یک جمع‌کننده نخست روی بخش‌های حقیقی و سپس روی بخش‌های موهومی کار می‌کند.

پیش از ورود به جمع‌کننده، هر عملوند هشت‌بیتی با بسط علامت[۶] به هفده بیت تبدیل می‌شود تا مقدارهای منفی در پهنای بزرگ‌تر درست بمانند. آشکارسازی سرریز اما روی همان هشت بیت اصلی انجام می‌گیرد، زیرا پرسش معنادار این است که آیا نتیجه در پهنای ورودی جا می‌شود یا نه. برای جمع، سرریز هنگامی رخ

[6] Sign Extension

می‌دهد که دو عملوند هم‌علامت باشند و علامت نتیجه با آن‌ها فرق کند؛ برای تفریق، هنگامی که دو عملوند ناهم‌علامت باشند و علامت نتیجه با عملوند نخست نخواند. پرچم سرریز نهایی، یای منطقی سرریز دو بخش است.

```
// Complex add/sub block for two signed complex numbers.
// I use one shared adder and calculate the real and imaginary parts in separate clock cycles.
module complex_addsub_mc #(
    parameter WIDTH = 8,
    parameter RESULT_WIDTH = 2*WIDTH + 1
) (
    input                           clk,
    input                           reset,
    input                           start,
    input                           sub,          // 0:add, 1:subtract
    input  signed [WIDTH-1:0]       a_real,
    input  signed [WIDTH-1:0]       a_imag,
    input  signed [WIDTH-1:0]       b_real,
    input  signed [WIDTH-1:0]       b_imag,
    output reg signed [RESULT_WIDTH-1:0] result_real,
    output reg signed [RESULT_WIDTH-1:0] result_imag,
    output reg                      valid,
    output reg                      overflow,
    output reg                      busy
);
    // Small FSM: latch inputs, calculate real part, calculate imaginary part, then raise valid.
    localparam S_IDLE   = 2'd0;
    localparam S_REAL   = 2'd1;
    localparam S_IMAG   = 2'd2;
    localparam S_DONE   = 2'd3;

    reg [1:0] state;
    reg op_reg;
    reg signed [WIDTH-1:0] ar, ai, br, bi;
    reg ovf_temp;

    reg  signed [RESULT_WIDTH-1:0] add_a, add_b;
    wire signed [RESULT_WIDTH-1:0] add_y;

    // The same add/sub hardware is reused for both components.
    shared_addsub #(.WIDTH(RESULT_WIDTH)) U_ADD (
        .a(add_a), .b(add_b), .sub(op_reg), .y(add_y)
    );

    // Sign extension keeps negative WIDTH-bit operands correct at the wider result width.
    function signed [RESULT_WIDTH-1:0] sext_in;
        input signed [WIDTH-1:0] x;
        begin
            sext_in = {{(RESULT_WIDTH-WIDTH){x[WIDTH-1]}}, x};
        end
    endfunction

    // Overflow is checked using the original WIDTH-bit signed result.
    function add_ovf;
        input signed [WIDTH-1:0] x;
        input signed [WIDTH-1:0] y;
        input signed [WIDTH-1:0] r;
        begin
            add_ovf = (~(x[WIDTH-1] ^ y[WIDTH-1])) & (r[WIDTH-1] ^ x[WIDTH-1]);
        end
    endfunction

    function sub_ovf;
        input signed [WIDTH-1:0] x;
        input signed [WIDTH-1:0] y;
        input signed [WIDTH-1:0] r;
        begin
            sub_ovf = (x[WIDTH-1] ^ y[WIDTH-1]) & (r[WIDTH-1] ^ x[WIDTH-1]);
        end
```

```
    endfunction

    // Select which component is connected to the shared adder in the current state.
    always @(*) begin
        add_a = 0;
        add_b = 0;
        case (state)
            S_REAL: begin add_a = sext_in(ar); add_b = sext_in(br); end
            S_IMAG: begin add_a = sext_in(ai); add_b = sext_in(bi); end
            default: begin add_a = 0; add_b = 0; end
        endcase
    end

    // Sequential part of the FSM. valid is only pulsed after both components are ready.
    always @(posedge clk) begin
        if (reset) begin
            state       <= S_IDLE;
            valid       <= 1'b0;
            busy        <= 1'b0;
            overflow    <= 1'b0;
            ovf_temp    <= 1'b0;
            result_real <= 0;
            result_imag <= 0;
            ar <= 0; ai <= 0; br <= 0; bi <= 0; op_reg <= 0;
        end else begin
            valid <= 1'b0;
            case (state)
                S_IDLE: begin
                    busy <= 1'b0;
                    if (start) begin
                        ar <= a_real; ai <= a_imag; br <= b_real; bi <= b_imag;
                        op_reg <= sub;
                        ovf_temp <= 1'b0;
                        overflow <= 1'b0;
                        busy <= 1'b1;
                        state <= S_REAL;
                    end
                end
                S_REAL: begin
                    result_real <= add_y;
                    if (op_reg)
                        ovf_temp <= sub_ovf(ar, br, add_y[WIDTH-1:0]);
                    else
                        ovf_temp <= add_ovf(ar, br, add_y[WIDTH-1:0]);
                    state <= S_IMAG;
                end
                S_IMAG: begin
                    result_imag <= add_y;
                    if (op_reg)
                        overflow <= ovf_temp | sub_ovf(ai, bi, add_y[WIDTH-1:0]);
                    else
                        overflow <= ovf_temp | add_ovf(ai, bi, add_y[WIDTH-1:0]);
                    state <= S_DONE;
                end
                S_DONE: begin
                    valid <= 1'b1;
                    busy  <= 1'b0;
                    state <= S_IDLE;
                end
            endcase
        end
    end
endmodule
```

کد ۲: پیمانه‌ی جمع و تفریق اعداد مختلط

ب) ضرب مختلط

این پیمانه هشت حالت دارد که چهار تای نخست آن‌ها چهار حاصل‌ضرب جزئی را یکی‌یکی می‌سازند و در ثبات نگه می‌دارند، و دو حالت بعدی با همان یک جمع‌کننده، تفریق بخش حقیقی و جمع بخش موهومی را انجام می‌دهند. بلوک ترکیبی انتخاب ورودی، در هر حالت عملوندهای درست را به ضرب‌کننده و جمع‌کننده می‌رساند؛ این بلوک در عمل همان تسهیم‌کننده‌های ورودی مسیر دادهٔ مشترک است.

```
// Multi-cycle complex multiplier.
// Instead of four multipliers, one multiplier is reused for ac, bd, ad and bc.
module complex_multiplier_mc #(
    parameter WIDTH = 8,
    parameter RESULT_WIDTH = 2*WIDTH + 1
) (
    input                          clk,
    input                          reset,
    input                          start,
    input  signed [WIDTH-1:0]      a_real,
    input  signed [WIDTH-1:0]      a_imag,
    input  signed [WIDTH-1:0]      b_real,
    input  signed [WIDTH-1:0]      b_imag,
    output reg signed [RESULT_WIDTH-1:0] result_real,
    output reg signed [RESULT_WIDTH-1:0] result_imag,
    output reg                     valid,
    output reg                     busy
);
    // States follow the calculation order: four products, real subtraction, imaginary addition, done.
    localparam S_IDLE = 4'd0, S_AC = 4'd1, S_BD = 4'd2,
               S_AD   = 4'd3, S_BC = 4'd4, S_REAL = 4'd5,
               S_IMAG = 4'd6, S_DONE = 4'd7;

    reg [3:0] state;
    reg signed [WIDTH-1:0] ar, ai, br, bi;
    reg signed [2*WIDTH-1:0] p_ac, p_bd, p_ad, p_bc;

    // Shared multiplier and shared add/sub datapath.
    reg  signed [WIDTH-1:0] mul_a, mul_b;
    wire signed [2*WIDTH-1:0] mul_y;
    reg  signed [RESULT_WIDTH-1:0] add_a, add_b;
    reg  add_sub;
    wire signed [RESULT_WIDTH-1:0] add_y;

    shared_multiplier #(.WIDTH(WIDTH)) U_MUL (
        .a(mul_a), .b(mul_b), .y(mul_y)
    );
    shared_addsub #(.WIDTH(RESULT_WIDTH)) U_ADD (
        .a(add_a), .b(add_b), .sub(add_sub), .y(add_y)
    );

    // Products are sign-extended before the final add/sub operation.
    function signed [RESULT_WIDTH-1:0] sext_prod;
        input signed [2*WIDTH-1:0] x;
        begin
            sext_prod = {{(RESULT_WIDTH-2*WIDTH){x[2*WIDTH-1]}}, x};
        end
    endfunction

    // Mux the required operands onto the shared units according to the current state.
    always @(*) begin
        mul_a = 0; mul_b = 0;
        add_a = 0; add_b = 0; add_sub = 1'b0;
        case (state)
            S_AC:   begin mul_a = ar; mul_b = br; end
            S_BD:   begin mul_a = ai; mul_b = bi; end
            S_AD:   begin mul_a = ar; mul_b = bi; end
            S_BC:   begin mul_a = ai; mul_b = br; end
```

```
            S_REAL: begin add_a = sext_prod(p_ac); add_b = sext_prod(p_bd); add_sub = 1'b1; end
            S_IMAG: begin add_a = sext_prod(p_ad); add_b = sext_prod(p_bc); add_sub = 1'b0; end
            default: ;
        endcase
    end

    // Store each partial product in a register and move to the next calculation step.
    always @(posedge clk) begin
        if (reset) begin
            state <= S_IDLE; valid <= 0; busy <= 0;
            result_real <= 0; result_imag <= 0;
            ar <= 0; ai <= 0; br <= 0; bi <= 0;
            p_ac <= 0; p_bd <= 0; p_ad <= 0; p_bc <= 0;
        end else begin
            valid <= 1'b0;
            case (state)
                S_IDLE: begin
                    busy <= 1'b0;
                    if (start) begin
                        ar <= a_real; ai <= a_imag; br <= b_real; bi <= b_imag;
                        busy <= 1'b1;
                        state <= S_AC;
                    end
                end
                S_AC:   begin p_ac <= mul_y; state <= S_BD; end
                S_BD:   begin p_bd <= mul_y; state <= S_AD; end
                S_AD:   begin p_ad <= mul_y; state <= S_BC; end
                S_BC:   begin p_bc <= mul_y; state <= S_REAL; end
                S_REAL: begin result_real <= add_y; state <= S_IMAG; end
                S_IMAG: begin result_imag <= add_y; state <= S_DONE; end
                S_DONE: begin valid <= 1'b1; busy <= 1'b0; state <= S_IDLE; end
            endcase
        end
    end
endmodule
```

کد ۳: پیمانه‌ی ضرب اعداد مختلط با یک ضرب‌کننده‌ی مشترک

ج) مزدوج و مربع اندازه

دو پیمانهٔ دیگر نیز با همین سبک ساخته شدند تا مجموعهٔ عملیات کامل شود. پیمانهٔ مزدوج، بخش حقیقی را دست‌نخورده نگه می‌دارد و بخش موهومی را منفی می‌کند؛ نکتهٔ آن این است که منفی‌سازی نیز به‌جای یک نفی‌کنندهٔ جداگانه، با همان جمع‌کنندهٔ مشترک و به‌صورت $0 - b$ انجام می‌شود تا قید دستورکار حتّی در این عمل ساده هم شکسته نشود.

```
// Complex conjugate block: keeps the real part and changes the sign of the imaginary part.
// Negation is implemented with the same add/sub style used in the rest of the design (0 - imag).
module complex_conjugate #(
    parameter WIDTH = 8,
    parameter RESULT_WIDTH = 2*WIDTH + 1
) (
    input                          clk,
    input                          reset,
    input                          start,
    input  signed [WIDTH-1:0]      a_real,
    input  signed [WIDTH-1:0]      a_imag,
    output reg signed [RESULT_WIDTH-1:0] result_real,
    output reg signed [RESULT_WIDTH-1:0] result_imag,
    output reg                     valid,
    output reg                     busy
);
```

```
    // A short FSM is used so the interface is the same start/busy/valid style as the other blocks.
    localparam S_IDLE = 2'd0, S_NEG = 2'd1, S_DONE = 2'd2;
    reg [1:0] state;
    reg signed [WIDTH-1:0] imag_reg;

    // Extend the imaginary input before negating it at RESULT_WIDTH.
    wire signed [RESULT_WIDTH-1:0] imag_ext;
    wire signed [RESULT_WIDTH-1:0] neg_imag;
    assign imag_ext = {{(RESULT_WIDTH-WIDTH){imag_reg[WIDTH-1]}}, imag_reg};

    shared_addsub #(.WIDTH(RESULT_WIDTH)) U_ADD (
        .a({RESULT_WIDTH{1'b0}}), .b(imag_ext), .sub(1'b1), .y(neg_imag)
    );

    // Inputs are accepted in IDLE and valid is pulsed for one clock in the DONE state.
    always @(posedge clk) begin
        if (reset) begin
            state <= S_IDLE; valid <= 0; busy <= 0;
            result_real <= 0; result_imag <= 0; imag_reg <= 0;
        end else begin
            valid <= 1'b0;
            case (state)
                S_IDLE: begin
                    busy <= 1'b0;
                    if (start) begin
                        result_real <= {{(RESULT_WIDTH-WIDTH){a_real[WIDTH-1]}}, a_real};
                        imag_reg <= a_imag;
                        busy <= 1'b1;
                        state <= S_NEG;
                    end
                end
                S_NEG: begin result_imag <= neg_imag; state <= S_DONE; end
                S_DONE: begin valid <= 1'b1; busy <= 1'b0; state <= S_IDLE; end
                default: state <= S_IDLE;
            endcase
        end
    end
endmodule
```

کد ۴: پیمانه‌ی مزدوج مختلط

پیمانهٔ مربع اندازه، مقدار $|z|^2 = a^2 + b^2$ را در سه چرخه محاسبه می‌کند: دو چرخه برای دو مربع با همان یک ضرب‌کننده، و یک چرخه برای جمع آن‌ها. چون نتیجه ذاتاً حقیقی است، بخش موهومی خروجی همواره صفر گذاشته می‌شود.

```
// Multi-cycle magnitude squared block: |z|^2 = real^2 + imag^2.
// One multiplier is reused for the two squares and the final result has zero imaginary part.
module magnitude_squared_mc #(
    parameter WIDTH = 8,
    parameter RESULT_WIDTH = 2*WIDTH + 1
) (
    input                           clk,
    input                           reset,
    input                           start,
    input  signed [WIDTH-1:0]       a_real,
    input  signed [WIDTH-1:0]       a_imag,
    output reg signed [RESULT_WIDTH-1:0] result,
    output reg                      valid,
    output reg                      busy
);
    // FSM order is a*a, b*b, sum, then valid.
    localparam S_IDLE = 3'd0, S_A2 = 3'd1, S_B2 = 3'd2,
               S_SUM  = 3'd3, S_DONE = 3'd4;
    reg [2:0] state;
    reg signed [WIDTH-1:0] ar, ai;
```

```
    reg signed [2*WIDTH-1:0] p_a2, p_b2;

    // Shared arithmetic signals for the two multiply cycles and the final addition.
    reg signed [WIDTH-1:0] mul_a, mul_b;
    wire signed [2*WIDTH-1:0] mul_y;
    wire signed [RESULT_WIDTH-1:0] a2_ext, b2_ext, sum_y;

    assign a2_ext = {{(RESULT_WIDTH-2*WIDTH){p_a2[2*WIDTH-1]}}, p_a2};
    assign b2_ext = {{(RESULT_WIDTH-2*WIDTH){p_b2[2*WIDTH-1]}}, p_b2};

    shared_multiplier #(.WIDTH(WIDTH)) U_MUL (
        .a(mul_a), .b(mul_b), .y(mul_y)
    );
    shared_addsub #(.WIDTH(RESULT_WIDTH)) U_ADD (
        .a(a2_ext), .b(b2_ext), .sub(1'b0), .y(sum_y)
    );

    // Choose a or b as the multiplier input depending on the current state.
    always @(*) begin
        mul_a = 0; mul_b = 0;
        case (state)
            S_A2: begin mul_a = ar; mul_b = ar; end
            S_B2: begin mul_a = ai; mul_b = ai; end
            default: ;
        endcase
    end

    // Save the two squared terms in registers before adding them.
    always @(posedge clk) begin
        if (reset) begin
            state <= S_IDLE; valid <= 0; busy <= 0; result <= 0;
            ar <= 0; ai <= 0; p_a2 <= 0; p_b2 <= 0;
        end else begin
            valid <= 1'b0;
            case (state)
                S_IDLE: begin
                    busy <= 1'b0;
                    if (start) begin
                        ar <= a_real; ai <= a_imag;
                        busy <= 1'b1;
                        state <= S_A2;
                    end
                end
                S_A2:   begin p_a2 <= mul_y; state <= S_B2; end
                S_B2:   begin p_b2 <= mul_y; state <= S_SUM; end
                S_SUM:  begin result <= sum_y; state <= S_DONE; end
                S_DONE: begin valid <= 1'b1; busy <= 1'b0; state <= S_IDLE; end
                default: state <= S_IDLE;
            endcase
        end
    end
endmodule
```

کد ۵: پیمانه‌ی مربع اندازه

۵.۸ واحد حساب و منطق یکپارچه

پس از آزمودن جداگانهٔ پیمانه‌ها، همهٔ عملیات در یک واحد حساب و منطق مختلط گرد آمدند. این واحد شانزده حالت دارد که هر حالت، یک گام از یکی از عملیات را نمایندگی می‌کند؛ برای نمونه چهار حالت برای چهار حاصل‌ضرب جزئی CMUL و دو حالت برای دو مؤلّفهٔ CADD.

نکتهٔ کلیدی معماری در همین‌جاست: در سراسر این واحد تنها یک نمونه از ضرب‌کننده و یک نمونه از

جمع‌کننده ساخته می‌شود و همهٔ شانزده حالت از همان دو نمونه استفاده می‌کنند. حالت جاری تعیین می‌کند که کدام عملوندها به ورودی این دو بلوک برسند. به این ترتیب، افزودن عملیات تازه به این واحد هزینهٔ حسابی اضافه‌ای ندارد و تنها چند حالت و چند مسیر تسهیم‌کننده می‌افزاید.

واحد حساب چهار پرچم وضعیّت نیز تولید می‌کند: سرریز، صفر بودن نتیجه، منفی بودن بخش حقیقی و منفی بودن بخش موهومی. هر چهار پرچم در حالت پایانی و هم‌زمان با نتیجه به‌روز می‌شوند تا هیچ‌گاه پرچمی متعلّق به عملیات پیشین در کنار نتیجهٔ تازه دیده نشود.

```
// Main complex ALU used by the processor.
// The FSM reuses one multiplier and one add/sub unit so CMUL and MAG2 are done over several cycles.
module complex_alu_mc #(
    parameter WIDTH = 8,
    parameter RESULT_WIDTH = 2*WIDTH + 1
) (
    input                           clk,
    input                           reset,
    input                           start,
    input      [2:0]                opcode,
    input  signed [WIDTH-1:0]       a_real,
    input  signed [WIDTH-1:0]       a_imag,
    input  signed [WIDTH-1:0]       b_real,
    input  signed [WIDTH-1:0]       b_imag,

    output reg signed [RESULT_WIDTH-1:0] result_real,
    output reg signed [RESULT_WIDTH-1:0] result_imag,
    output reg                      valid,
    output reg                      overflow,
    output reg                      zero,
    output reg                      real_negative,
    output reg                      imag_negative,
    output reg                      busy
);
    // Opcodes used by the instruction decoder.
    localparam OP_CADD = 3'b000,
               OP_CSUB = 3'b001,
               OP_CMUL = 3'b010,
               OP_CONJ = 3'b011,
               OP_MAG2 = 3'b100,
               OP_NOP  = 3'b101;

    // One state is used for each step that needs the shared arithmetic hardware.
    localparam ST_IDLE      = 5'd0,
               ST_ADD_R     = 5'd1,
               ST_ADD_I     = 5'd2,
               ST_SUB_R     = 5'd3,
               ST_SUB_I     = 5'd4,
               ST_MUL_AC    = 5'd5,
               ST_MUL_BD    = 5'd6,
               ST_MUL_AD    = 5'd7,
               ST_MUL_BC    = 5'd8,
               ST_MUL_REAL  = 5'd9,
               ST_MUL_IMAG  = 5'd10,
               ST_CONJ_NEG  = 5'd11,
               ST_MAG_A2    = 5'd12,
               ST_MAG_B2    = 5'd13,
               ST_MAG_SUM   = 5'd14,
               ST_FINISH    = 5'd15;

    // Input and intermediate registers are kept because some operations need multiple clocks.
    reg [4:0] state;
    reg [2:0] op_reg;
    reg signed [WIDTH-1:0] ar, ai, br, bi;
    reg signed [2*WIDTH-1:0] p_ac, p_bd, p_ad, p_bc, p_a2, p_b2;
    reg signed [RESULT_WIDTH-1:0] temp_real, temp_imag;
    reg overflow_temp;
```

```
    // Only one multiplier is instantiated. Its inputs change with the FSM state.
    reg  signed [WIDTH-1:0] mul_a, mul_b;
    wire signed [2*WIDTH-1:0] mul_y;
    shared_multiplier #(.WIDTH(WIDTH)) U_SHARED_MUL (
        .a(mul_a), .b(mul_b), .y(mul_y)
    );

    // The same add/sub unit is used for CADD, CSUB, the final CMUL sums and MAG2.
    reg  signed [RESULT_WIDTH-1:0] add_a, add_b;
    reg                            add_sub;
    wire signed [RESULT_WIDTH-1:0] add_y;
    shared_addsub #(.WIDTH(RESULT_WIDTH)) U_SHARED_ADD (
        .a(add_a), .b(add_b), .sub(add_sub), .y(add_y)
    );

    // Helper functions keep the signed values at a common result width.
    function signed [RESULT_WIDTH-1:0] sext_in;
        input signed [WIDTH-1:0] x;
        begin
            sext_in = {{(RESULT_WIDTH-WIDTH){x[WIDTH-1]}}, x};
        end
    endfunction

    function signed [RESULT_WIDTH-1:0] sext_prod;
        input signed [2*WIDTH-1:0] x;
        begin
            sext_prod = {{(RESULT_WIDTH-2*WIDTH){x[2*WIDTH-1]}}, x};
        end
    endfunction

    // Signed overflow logic is only needed for the WIDTH-bit add/sub operations.
    function add_ovf;
        input signed [WIDTH-1:0] x;
        input signed [WIDTH-1:0] y;
        input signed [WIDTH-1:0] r;
        begin
            add_ovf = (~(x[WIDTH-1] ^ y[WIDTH-1])) & (r[WIDTH-1] ^ x[WIDTH-1]);
        end
    endfunction

    function sub_ovf;
        input signed [WIDTH-1:0] x;
        input signed [WIDTH-1:0] y;
        input signed [WIDTH-1:0] r;
        begin
            sub_ovf = (x[WIDTH-1] ^ y[WIDTH-1]) & (r[WIDTH-1] ^ x[WIDTH-1]);
        end
    endfunction

    // This combinational block works like the input muxes of the shared datapath.
    always @(*) begin
        mul_a   = 0;
        mul_b   = 0;
        add_a   = 0;
        add_b   = 0;
        add_sub = 1'b0;

        case (state)
            ST_ADD_R: begin add_a = sext_in(ar); add_b = sext_in(br); add_sub = 1'b0; end
            ST_ADD_I: begin add_a = sext_in(ai); add_b = sext_in(bi); add_sub = 1'b0; end
            ST_SUB_R: begin add_a = sext_in(ar); add_b = sext_in(br); add_sub = 1'b1; end
            ST_SUB_I: begin add_a = sext_in(ai); add_b = sext_in(bi); add_sub = 1'b1; end

            ST_MUL_AC: begin mul_a = ar; mul_b = br; end
            ST_MUL_BD: begin mul_a = ai; mul_b = bi; end
            ST_MUL_AD: begin mul_a = ar; mul_b = bi; end
            ST_MUL_BC: begin mul_a = ai; mul_b = br; end
            ST_MUL_REAL: begin add_a = sext_prod(p_ac); add_b = sext_prod(p_bd); add_sub = 1'b1; end
            ST_MUL_IMAG: begin add_a = sext_prod(p_ad); add_b = sext_prod(p_bc); add_sub = 1'b0; end

            ST_CONJ_NEG: begin add_a = 0; add_b = sext_in(ai); add_sub = 1'b1; end
```

```
            ST_MAG_A2: begin mul_a = ar; mul_b = ar; end
            ST_MAG_B2: begin mul_a = ai; mul_b = ai; end
            ST_MAG_SUM: begin add_a = sext_prod(p_a2); add_b = sext_prod(p_b2); add_sub = 1'b0; end
            default: ;
        endcase
    end

    // Main controller: start latches the operands, then the FSM advances one step per clock.
    always @(posedge clk) begin
        if (reset) begin
            state         <= ST_IDLE;
            op_reg        <= OP_NOP;
            ar <= 0; ai <= 0; br <= 0; bi <= 0;
            p_ac <= 0; p_bd <= 0; p_ad <= 0; p_bc <= 0; p_a2 <= 0; p_b2 <= 0;
            temp_real     <= 0;
            temp_imag     <= 0;
            overflow_temp <= 1'b0;
            result_real   <= 0;
            result_imag   <= 0;
            valid         <= 1'b0;
            overflow      <= 1'b0;
            zero          <= 1'b1;
            real_negative <= 1'b0;
            imag_negative <= 1'b0;
            busy          <= 1'b0;
        end else begin
            valid <= 1'b0;

            case (state)
                ST_IDLE: begin
                    busy <= 1'b0;
                    if (start) begin
                        ar <= a_real; ai <= a_imag; br <= b_real; bi <= b_imag;
                        op_reg <= opcode;
                        temp_real <= 0;
                        temp_imag <= 0;
                        overflow_temp <= 1'b0;
                        overflow <= 1'b0;
                        busy <= 1'b1;

                        case (opcode)
                            OP_CADD: state <= ST_ADD_R;
                            OP_CSUB: state <= ST_SUB_R;
                            OP_CMUL: state <= ST_MUL_AC;
                            OP_CONJ: begin
                                temp_real <= sext_in(a_real);
                                state <= ST_CONJ_NEG;
                            end
                            OP_MAG2: begin
                                temp_imag <= 0;
                                state <= ST_MAG_A2;
                            end
                            default: begin
                                temp_real <= 0;
                                temp_imag <= 0;
                                state <= ST_FINISH;
                            end
                        endcase
                    end
                end

                ST_ADD_R: begin
                    temp_real <= add_y;
                    overflow_temp <= add_ovf(ar, br, add_y[WIDTH-1:0]);
                    state <= ST_ADD_I;
                end
                ST_ADD_I: begin
                    temp_imag <= add_y;
                    overflow_temp <= overflow_temp | add_ovf(ai, bi, add_y[WIDTH-1:0]);
                    state <= ST_FINISH;
                end
```

```
                ST_SUB_R: begin
                    temp_real <= add_y;
                    overflow_temp <= sub_ovf(ar, br, add_y[WIDTH-1:0]);
                    state <= ST_SUB_I;
                end
                ST_SUB_I: begin
                    temp_imag <= add_y;
                    overflow_temp <= overflow_temp | sub_ovf(ai, bi, add_y[WIDTH-1:0]);
                    state <= ST_FINISH;
                end

                // Complex multiply: ac, bd, ad and bc are calculated one after another.
                ST_MUL_AC: begin p_ac <= mul_y; state <= ST_MUL_BD; end
                ST_MUL_BD: begin p_bd <= mul_y; state <= ST_MUL_AD; end
                ST_MUL_AD: begin p_ad <= mul_y; state <= ST_MUL_BC; end
                ST_MUL_BC: begin p_bc <= mul_y; state <= ST_MUL_REAL; end
                ST_MUL_REAL: begin temp_real <= add_y; state <= ST_MUL_IMAG; end
                ST_MUL_IMAG: begin temp_imag <= add_y; state <= ST_FINISH; end

                ST_CONJ_NEG: begin
                    temp_imag <= add_y;
                    state <= ST_FINISH;
                end

                // Magnitude squared also reuses the multiplier: a*a, b*b, then one addition.
                ST_MAG_A2: begin p_a2 <= mul_y; state <= ST_MAG_B2; end
                ST_MAG_B2: begin p_b2 <= mul_y; state <= ST_MAG_SUM; end
                ST_MAG_SUM: begin
                    temp_real <= add_y;
                    temp_imag <= 0;
                    state <= ST_FINISH;
                end

                // Results and status flags are updated together when the operation is complete.
                ST_FINISH: begin
                    result_real   <= temp_real;
                    result_imag   <= temp_imag;
                    overflow      <= overflow_temp;
                    zero          <= (temp_real == 0) && (temp_imag == 0);
                    real_negative <= temp_real[RESULT_WIDTH-1];
                    imag_negative <= temp_imag[RESULT_WIDTH-1];
                    valid         <= 1'b1;
                    busy          <= 1'b0;
                    state         <= ST_IDLE;
                end

                default: state <= ST_IDLE;
            endcase
        end
    end
endmodule
```

کد ۶: واحد حساب و منطق مختلط با منابع مشترک

۶.۸ حافظه‌ی دستور و واحد کنترل

حافظهٔ دستور یک آرایهٔ ۳۲ کلمه‌ای است که محتوای آن در خود توصیف سخت‌افزار مقداردهی می‌شود. این انتخاب عمدی است: با آن، ModelSim و Quartus هر دو دقیقاً همان برنامه را می‌بینند و نیازی به پروندهٔ جداگانهٔ مقداردهی حافظه نیست. خانه‌های استفاده‌نشده با دستور بدون عملیات پر می‌شوند تا هیچ خانه‌ای نامعیّن نماند.

```verilog
// Simple 32-word instruction memory used by the processor.
// I initialize the test program in Verilog so ModelSim and Quartus use the same instruction contents.
module instruction_memory #(
    parameter WIDTH = 8,
    parameter DEPTH = 32,
    parameter WORD_WIDTH = 4*WIDTH + 3
) (
    input      [4:0] address,
    output     [WORD_WIDTH-1:0] data
);
    localparam OP_CADD = 3'b000,
               OP_CSUB = 3'b001,
               OP_CMUL = 3'b010,
               OP_CONJ = 3'b011,
               OP_MAG2 = 3'b100,
               OP_NOP  = 3'b101;

    reg [WORD_WIDTH-1:0] mem [0:DEPTH-1];
    integer i;

    // Pack opcode and four signed operand fields into one memory word.
    function [WORD_WIDTH-1:0] pack;
        input [2:0] op;
        input signed [WIDTH-1:0] ar;
        input signed [WIDTH-1:0] ai;
        input signed [WIDTH-1:0] br;
        input signed [WIDTH-1:0] bi;
        begin
            pack = {op, ar, ai, br, bi};
        end
    endfunction

    // Unused locations are cleared first, then the test instructions are written below.
    initial begin
        for (i = 0; i < DEPTH; i = i + 1)
            mem[i] = pack(OP_NOP, 0, 0, 0, 0);

        // test program used in the final processor testbench
        mem[0] = pack(OP_CADD,   3,   4,   5,   2);   // (3+j4)+(5+j2) = 8+j6
        mem[1] = pack(OP_CADD,  -3,  -4,  -5,  -2);   // negative + negative = -8-j6
        mem[2] = pack(OP_CSUB,   2,   3,   7,   8);   // = -5-j5
        mem[3] = pack(OP_CMUL,   2,   3,  -4,   5);   // = -23-j2
        mem[4] = pack(OP_CONJ,  -6,   7,   0,   0);   // = -6-j7
        mem[5] = pack(OP_MAG2,   3,  -4,   0,   0);   // = 25+j0
        mem[6] = pack(OP_CADD,   5,  -5,  -5,   5);   // = 0+j0 (zero flag)
        mem[7] = pack(OP_CADD, 127,   0,   1,   0);   // = 128+j0 (8-bit overflow flag)
    end

    assign data = mem[address];
endmodule
```

کد ۷: حافظه‌ی دستور ۳۲ کلمه‌ای و برنامه‌ی آزمون

واحد کنترل، هفت حالت دارد که به‌ترتیب عبارت‌اند از بی‌کار، واکشی[۷] دستور، رمزگشایی[۸] کد عملیات، خواندن میدان‌های عملوند، صدور پالس شروع به واحد حساب، انتظار تا پایان عملیات، و بازنویسی[۹] نتیجه و پرچم‌ها. شمارندهٔ برنامه پس از هر بازنویسی یک واحد جلو می‌رود و با رسیدن به پایان برنامه، پرچم اتمام بالا می‌رود.

[7] Fetch
[8] Decode
[9] Write-Back

```
// Top-level controller for the Lab 8 processor.
// It reads an instruction from memory, decodes the operands, starts the complex ALU and writes back
     the result.
module complex_processor #(
    parameter WIDTH = 8,
    parameter RESULT_WIDTH = 2*WIDTH + 1,
    parameter PROGRAM_LENGTH = 8
) (
    input                           clk,
    input                           reset,
    input                           start,

    output reg signed [RESULT_WIDTH-1:0] result_real,
    output reg signed [RESULT_WIDTH-1:0] result_imag,
    output reg                      valid,
    output reg                      overflow,
    output reg                      zero,
    output reg                      real_negative,
    output reg                      imag_negative,
    output reg [4:0]                result_pc,
    output                          busy,
    output reg                      done
);
    // Instruction format: opcode + real/imag fields for operand A and operand B.
    localparam INSTR_WIDTH = 4*WIDTH + 3;

    // Processor FSM stages. P_WAIT is important because some ALU instructions take several clocks.
    localparam P_IDLE   = 4'd0,
               P_FETCH  = 4'd1,
               P_DECODE = 4'd2,
               P_READ   = 4'd3,
               P_START  = 4'd4,
               P_WAIT   = 4'd5,
               P_WB     = 4'd6;

    // Registers between stages keep the fetched instruction and decoded operands stable.
    reg [3:0] state;
    reg [4:0] pc;
    reg [4:0] fetched_pc;
    wire [INSTR_WIDTH-1:0] instruction;
    reg  [INSTR_WIDTH-1:0] instruction_reg;

    reg [2:0] opcode_reg;
    reg signed [WIDTH-1:0] a_real_reg, a_imag_reg, b_real_reg, b_imag_reg;

    wire alu_start;
    wire signed [RESULT_WIDTH-1:0] alu_real, alu_imag;
    wire alu_valid, alu_overflow, alu_zero, alu_real_negative, alu_imag_negative, alu_busy;

    // Program memory is 32 words deep; PC selects the current instruction.
    instruction_memory #(
        .WIDTH(WIDTH), .DEPTH(32), .WORD_WIDTH(INSTR_WIDTH)
    ) IMEM (
        .address(pc), .data(instruction)
    );

    assign alu_start = (state == P_START);

    // The processor sends one start pulse to the ALU and waits for alu_valid before write-back.
    complex_alu_mc #(
        .WIDTH(WIDTH), .RESULT_WIDTH(RESULT_WIDTH)
    ) ALU (
        .clk(clk), .reset(reset), .start(alu_start), .opcode(opcode_reg),
        .a_real(a_real_reg), .a_imag(a_imag_reg),
        .b_real(b_real_reg), .b_imag(b_imag_reg),
        .result_real(alu_real), .result_imag(alu_imag),
        .valid(alu_valid), .overflow(alu_overflow), .zero(alu_zero),
        .real_negative(alu_real_negative), .imag_negative(alu_imag_negative),
        .busy(alu_busy)
    );

    assign busy = (state != P_IDLE);
```

```
72
73      // Synchronous processor control and output registers.
74      always @(posedge clk) begin
75          if (reset) begin
76              state <= P_IDLE;
77              pc <= 0;
78              fetched_pc <= 0;
79              instruction_reg <= 0;
80              opcode_reg <= 0;
81              a_real_reg <= 0; a_imag_reg <= 0; b_real_reg <= 0; b_imag_reg <= 0;
82              result_real <= 0; result_imag <= 0;
83              valid <= 0; overflow <= 0; zero <= 1;
84              real_negative <= 0; imag_negative <= 0;
85              result_pc <= 0;
86              done <= 0;
87          end else begin
88              valid <= 1'b0;
89
90              case (state)
91                  P_IDLE: begin
92                      if (start) begin
93                          pc <= 0;
94                          done <= 1'b0;
95                          state <= P_FETCH;
96                      end
97                  end
98
99                  // fetch instruction
100                 P_FETCH: begin
101                     instruction_reg <= instruction;
102                     fetched_pc <= pc;
103                     state <= P_DECODE;
104                 end
105
106                 // decode opcode
107                 P_DECODE: begin
108                     opcode_reg <= instruction_reg[4*WIDTH+2:4*WIDTH];
109                     state <= P_READ;
110                 end
111
112                 // read operand fields
113                 P_READ: begin
114                     a_real_reg <= instruction_reg[4*WIDTH-1:3*WIDTH];
115                     a_imag_reg <= instruction_reg[3*WIDTH-1:2*WIDTH];
116                     b_real_reg <= instruction_reg[2*WIDTH-1:WIDTH];
117                     b_imag_reg <= instruction_reg[WIDTH-1:0];
118                     state <= P_START;
119                 end
120
121                 // send a one-clock start pulse to the ALU
122                 P_START: begin
123                     state <= P_WAIT;
124                 end
125
126                 // wait here until the current ALU operation is finished
127                 P_WAIT: begin
128                     if (alu_valid)
129                         state <= P_WB;
130                 end
131
132                 // write result and flags to the outputs
133                 P_WB: begin
134                     result_real   <= alu_real;
135                     result_imag   <= alu_imag;
136                     overflow      <= alu_overflow;
137                     zero          <= alu_zero;
138                     real_negative <= alu_real_negative;
139                     imag_negative <= alu_imag_negative;
140                     result_pc     <= fetched_pc;
141                     valid         <= 1'b1;
142
143                     if (fetched_pc == PROGRAM_LENGTH-1) begin
```

```
144                         done <= 1'b1;
145                         state <= P_IDLE;
146                     end else begin
147                         pc <= pc + 1'b1;
148                         state <= P_FETCH;
149                     end
150                 end
151 
152                 default: state <= P_IDLE;
153             endcase
154         end
155     end
156 endmodule
```

کد ۸: واحد کنترل و اتّصال حافظه به واحد حساب

نکته‌ای درباره‌ی خواسته‌ی «پایپ‌لاین»

دستورکار در بند سوم خود، از «یک واحد پایپ‌لاین» سخن گفته است که دستورها را از حافظه بخواند و عملیات را به‌صورت پایپ‌لاین انجام دهد. لازم است در اینجا صریح باشیم: طرح حاضر دارای *مراحل* جدا از هم به سبک پایپ‌لاین است — واکشی، رمزگشایی، خواندن عملوند، اجرا و بازنویسی — امّا این مراحل هم*پوشانی* ندارند؛ یعنی دستور بعدی تنها پس از پایان کامل دستور جاری واکشی می‌شود. بنابراین طرح از نوع چندچرخه‌ای با مراحل مشخّص است و نه پایپ‌لاین با اجرای هم‌پوشان.

این انتخاب، پیامد مستقیم قید دیگر همان صفحهٔ دستورکار است. پایپ‌لاین واقعی زمانی معنا دارد که چند دستور هم‌زمان در مراحل مختلف باشند؛ امّا وقتی تنها یک ضرب‌کننده و یک جمع‌کننده در کلّ طرح مجاز است، دو دستور حسابی نمی‌توانند هم‌زمان در مرحلهٔ اجرا باشند، زیرا هر دو به همان یک ضرب‌کننده نیاز دارند. رسیدن به هم‌پوشانی واقعی یا نیازمند تکثیر واحدهای حسابی است — که قید دستورکار آن را منع می‌کند — یا نیازمند منطق توقّف و مدیریت مخاطرهٔ ساختاری[۱۰] است که در عمل، بهرهٔ پایپ‌لاین را برای این مجموعهٔ دستور از میان می‌برد. میان این دو خواسته، قید صریح و کمّی منابع بر توصیف کیفی پایپ‌لاین ترجیح داده شد.

۷.۸ میزهای آزمون

برای هر پیمانه یک میز آزمون خودآزمای جداگانه نوشته شد و در پایان، یک میز آزمون یکپارچه کلّ رایانه را می‌آزماید. جدول زیر پوشش هر یک را خلاصه می‌کند.

[10] Structural Hazard

میز آزمون	شمار وارسی	آنچه سنجیده می‌شود
جمع و تفریق	4	جمع مثبت، جمع منفی، تفریق، و حالت سرریز
ضرب	3	ضرب با نتیجه‌ی منفی، ضرب بزرگ، ضرب در عدد حقیقی
مزدوج	2	مزدوج با بخش موهومی مثبت و با بخش موهومی منفی
مربع اندازه	3	مقدار متعارف، مقدار بزرگ، و ورودی صفر
واحد حساب	8	هر پنج عملیات به‌همراه پرچم‌های صفر و سرریز
رایانه‌ی کامل	8	اجرای کامل برنامه‌ی حافظه و وارسی نتیجه‌ی هر دستور

جدول ۲: پوشش شش میز آزمون آزمایش

میز آزمون رایانهٔ کامل شیوهٔ متفاوتی دارد که ارزش توضیح دارد. این میز به‌جای انتظار برای زمان‌های از پیش تعیین‌شده، در هر لبهٔ پایین‌رونده پرچم اعتبار را می‌پاید و هرگاه نتیجه‌ای اعلام شود، با استفاده از شمارندهٔ برنامهٔ همراه آن نتیجه تشخیص می‌دهد که کدام دستور تمام شده و آن را با مقدار انتظاری خود می‌سنجد. این کار باعث می‌شود آزمون به طول متغیّر عملیات‌ها وابسته نباشد —— که در این طرح واقعاً متغیّر است، چون CADD کوتاه و CMUL بلند است —— و در پایان نیز بررسی می‌کند که دقیقاً هشت نتیجه دیده شده باشد.

```
`timescale 1ns/1ps


// Final testbench for the whole processor.
// The instruction memory contains eight operations, and every valid result is checked using its PC.
module tb_complex_processor;
    localparam WIDTH=8;
    localparam RW=2*WIDTH+1;

    reg clk,reset,start;
    wire signed [RW-1:0] rr,ri;
    wire valid,overflow,zero,real_negative,imag_negative,busy,done;
    wire [4:0] result_pc;
    integer errors, seen;

    complex_processor #(.WIDTH(WIDTH),.RESULT_WIDTH(RW),.PROGRAM_LENGTH(8)) dut(
        .clk(clk),.reset(reset),.start(start),
        .result_real(rr),.result_imag(ri),
        .valid(valid),.overflow(overflow),.zero(zero),
        .real_negative(real_negative),.imag_negative(imag_negative),
        .result_pc(result_pc),.busy(busy),.done(done)
    );

    always #5 clk=~clk;

    // This task compares the complex result and all four status flags for one completed instruction.
    task check_result;
        input [4:0] pc;
        input integer exp_r,exp_i;
        input exp_ovf,exp_zero,exp_nr,exp_ni;
        begin
            $display("------------------------------------------------------------");
            $display("Instruction PC=%0d completed",pc);
            $display("expected = (%0d,%0d) flags O/Z/NR/NI=%b%b%b%b",
                     exp_r,exp_i,exp_ovf,exp_zero,exp_nr,exp_ni);
            $display("got      = (%0d,%0d) flags O/Z/NR/NI=%b%b%b%b",
                     $signed(rr),$signed(ri),overflow,zero,real_negative,imag_negative);

            if(($signed(rr)!==exp_r)||($signed(ri)!==exp_i)||
```

```
                (overflow!==exp_ovf)||(zero!==exp_zero)||
                (real_negative!==exp_nr)||(imag_negative!==exp_ni)) begin
                 $display("RESULT   : FAIL");
                 errors=errors+1;
            end else begin
                 $display("RESULT   : PASS");
            end
            seen=seen+1;
        end
    endtask

    // Sample on the falling edge so values assigned at the previous rising edge are already stable.
    always @(negedge clk) begin
        if(valid) begin
            case(result_pc)
                0: check_result(result_pc,  8,  6, 0,0,0,0); // CADD
                1: check_result(result_pc, -8, -6, 0,0,1,1); // CADD negative
                2: check_result(result_pc, -5, -5, 0,0,1,1); // CSUB
                3: check_result(result_pc,-23, -2, 0,0,1,1); // CMUL
                4: check_result(result_pc, -6, -7, 0,0,1,1); // CONJ
                5: check_result(result_pc, 25,  0, 0,0,0,0); // MAG2
                6: check_result(result_pc,  0,  0, 0,1,0,0); // zero flag
                7: check_result(result_pc,128,  0, 1,0,0,0); // overflow flag
                default: begin
                    $display("FAIL : unexpected result_pc=%0d",result_pc);
                    errors=errors+1;
                end
            endcase
        end
    end

    // Start the processor once and wait until the complete memory program has finished.
    initial begin
        clk=0;reset=1;start=0;
        errors=0;seen=0;

        $display("============================================================");
        $display("      Lab 8 - Complete Processor Testbench");
        $display("============================================================");
        $display("The memory contains 8 test instructions.");

        // reset first, then one clock start pulse
        repeat(3) @(negedge clk);
        reset=0;
        @(negedge clk); start=1;
        @(negedge clk); start=0;

        wait(done==1'b1);
        repeat(2) @(negedge clk);

        if(seen!=8) begin
            $display("FAIL : expected 8 completed instructions, saw %0d",seen);
            errors=errors+1;
        end

        $display("============================================================");
        if(errors==0)
            $display("FINAL RESULT : ALL 8 PROCESSOR TESTS PASSED");
        else
            $display("FINAL RESULT : PROCESSOR HAS %0d ERROR(S)",errors);
        $display("============================================================");
        $finish;
    end
endmodule
```

کد ۹: میز آزمون یکپارچه‌ی رایانه‌ی اعداد مختلط

۸.۸ اجرا در ModelSim و تحلیل نتایج

هر آزمون با پروندهٔ فرمان جداگانهٔ خود اجرا می‌شود تا شکل موج هر پیمانه مستقل بررسی و ثبت گردد. فهرست کامل این فرمان‌ها در پرونده‌ای در کنار پروژه آمده و ترتیب اجرای آن‌ها چنین است: جمع و تفریق، ضرب، مزدوج، مربع اندازه، واحد حساب، و سرانجام رایانهٔ کامل.

```
# Lab 8 - final processor simulation
# Run this file from the modelsim folder with: do run_all.do

transcript on
if {[file exists work]} { vdel -lib work -all }
vlib work
vmap work work

# compile RTL
vlog ../rtl/shared_arith.v
vlog ../rtl/complex_addsub_mc.v
vlog ../rtl/complex_multiplier_mc.v
vlog ../rtl/complex_conjugate.v
vlog ../rtl/magnitude_squared_mc.v
vlog ../rtl/complex_alu_mc.v
vlog ../rtl/instruction_memory.v
vlog ../rtl/complex_processor.v

# compile testbench
vlog ../tb/tb_complex_processor.v

vsim -voptargs=+acc work.tb_complex_processor
view wave
delete wave *

# main input/control signals
add wave -divider {TOP CONTROL}
add wave sim:/tb_complex_processor/clk
add wave sim:/tb_complex_processor/reset
add wave sim:/tb_complex_processor/start
add wave sim:/tb_complex_processor/busy
add wave sim:/tb_complex_processor/done

# results and flags
add wave -divider {RESULT AND FLAGS}
add wave -radix unsigned  sim:/tb_complex_processor/result_pc
add wave -radix decimal   sim:/tb_complex_processor/rr
add wave -radix decimal   sim:/tb_complex_processor/ri
add wave sim:/tb_complex_processor/valid
add wave sim:/tb_complex_processor/overflow
add wave sim:/tb_complex_processor/zero
add wave sim:/tb_complex_processor/real_negative
add wave sim:/tb_complex_processor/imag_negative

# processor controller
add wave -divider {PROCESSOR FSM}
add wave -radix unsigned sim:/tb_complex_processor/dut/pc
add wave sim:/tb_complex_processor/dut/state
add wave sim:/tb_complex_processor/dut/opcode_reg

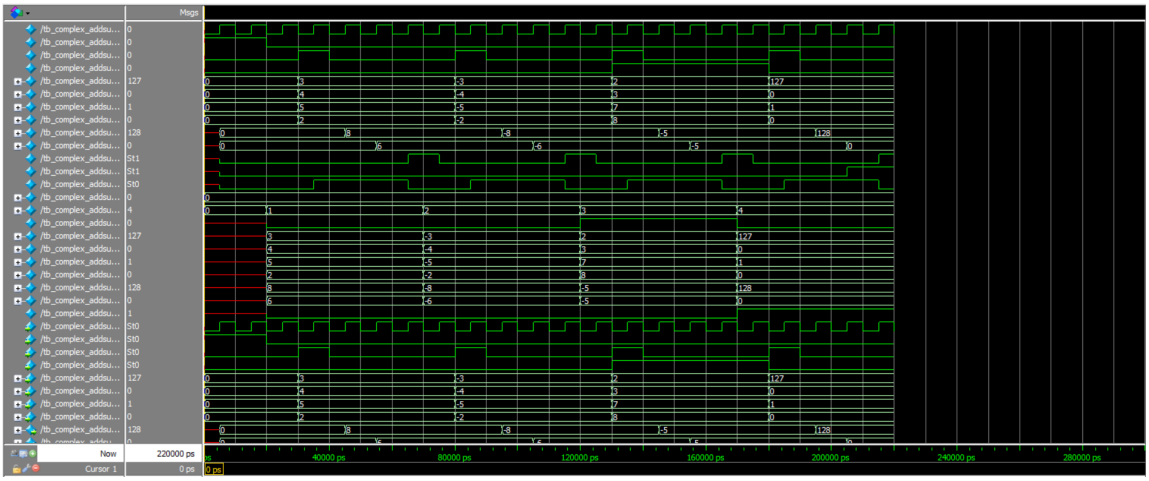
# ALU states and the shared resources
add wave -divider {ALU FSM AND SHARED UNITS}
add wave sim:/tb_complex_processor/dut/ALU/state
add wave sim:/tb_complex_processor/dut/ALU/busy
add wave sim:/tb_complex_processor/dut/ALU/valid
add wave -radix decimal sim:/tb_complex_processor/dut/ALU/mul_a
add wave -radix decimal sim:/tb_complex_processor/dut/ALU/mul_b
add wave -radix decimal sim:/tb_complex_processor/dut/ALU/mul_y
add wave -radix decimal sim:/tb_complex_processor/dut/ALU/add_a
add wave -radix decimal sim:/tb_complex_processor/dut/ALU/add_b
```

```
add wave sim:/tb_complex_processor/dut/ALU/add_sub
add wave -radix decimal sim:/tb_complex_processor/dut/ALU/add_y

# set the useful time range before running, so the final wave is not zoomed on 1 ns
WaveRestoreZoom {0 ns} {850 ns}
run -all
```

کد ۱۰: پرونده‌ی فرمان اجرای شبیه‌سازی رایانه‌ی کامل

الف) پیمانه‌ی جمع و تفریق



شکل ۱: شکل موج پیمانه‌ی جمع و تفریق مختلط

```
# Reading modelsim.ini
ModelSim> do run_addsub.do
# if {[file exists work]} { vdel -lib work -all }
# vlib work
# vmap work work
# Modifying modelsim.ini
# vlog ../rtl/shared_arith.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module shared_addsub
# -- Compiling module shared_multiplier
#
# Top level modules:
#       shared_addsub
#       shared_multiplier
# vlog ../rtl/complex_addsub_mc.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module complex_addsub_mc
#
# Top level modules:
```

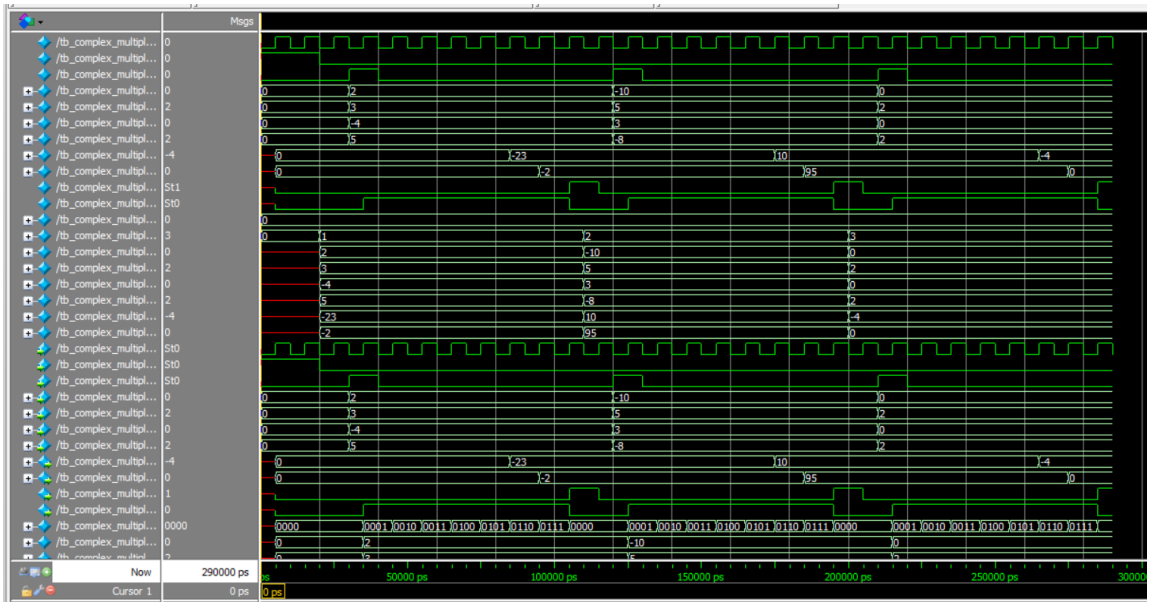
شکل ۲: رونوشت آزمون جمع و تفریق

```
# run -all
# ============================================================
#      Lab 8 - Complex Add/Sub Testbench
# ============================================================
# ------------------------------------------------------------
# TEST 1 : COMPLEX ADD
# A = (3,4)   B = (5,2)
# expected = (8,6), overflow=0
# got      = (8,6), overflow=0
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 2 : COMPLEX ADD
# A = (-3,-4)   B = (-5,-2)
# expected = (-8,-6), overflow=0
# got      = (-8,-6), overflow=0
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 3 : COMPLEX SUB
# A = (2,3)   B = (7,8)
# expected = (-5,-5), overflow=0
# got      = (-5,-5), overflow=0
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 4 : COMPLEX ADD
# A = (127,0)   B = (1,0)
# expected = (128,0), overflow=1
# got      = (128,0), overflow=1
# RESULT   : PASS
# ============================================================
# ADD/SUB SUMMARY : ALL 4 TESTS PASSED
# ============================================================
# ** Note: $finish    : ../tb/tb_complex_addsub_mc.v(97)
#    Time: 220 ns  Iteration: 1  Instance: /tb_complex_addsub_mc
```

شکل ۳: ادامه‌ی رونوشت و گزارش قبولی چهار وارسی

در شکل موج، الگوی چهارحالتی ماشین به‌روشنی دیده می‌شود: با هر پالس شروع، پرچم اشغال بالا می‌رود، دو چرخه بعد بخش‌های حقیقی و موهومی نتیجه یکی‌یکی نوشته می‌شوند و سپس پرچم اعتبار برای یک چرخه بالا می‌آید. وارسی چهارم، جمع $127 + 1$ را می‌آزماید که در هشت بیت علامت‌دار جا نمی‌شود؛ نتیجهٔ هفده‌بیتی مقدار درست 128 را نگه می‌دارد و هم‌زمان پرچم سرریز بالا می‌رود تا اعلام کند این مقدار در پهنای ورودی نمایش‌پذیر نیست.

ب) پیمانه‌ی ضرب



شکل ۴: شکل موج پیمانه‌ی ضرب مختلط

```
VSIM(paused)> do run_multiplier.do
# Lab 8 - complex multiplier test
# transcript on
# if {[file exists work]} { vdel -lib work -all }
# vlib work
# vmap work work
# Modifying modelsim.ini
# vlog ../rtl/shared_arith.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module shared_addsub
# -- Compiling module shared_multiplier
#
# Top level modules:
#       shared_addsub
#       shared_multiplier
# vlog ../rtl/complex_multiplier_mc.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module complex_multiplier_mc
#
```

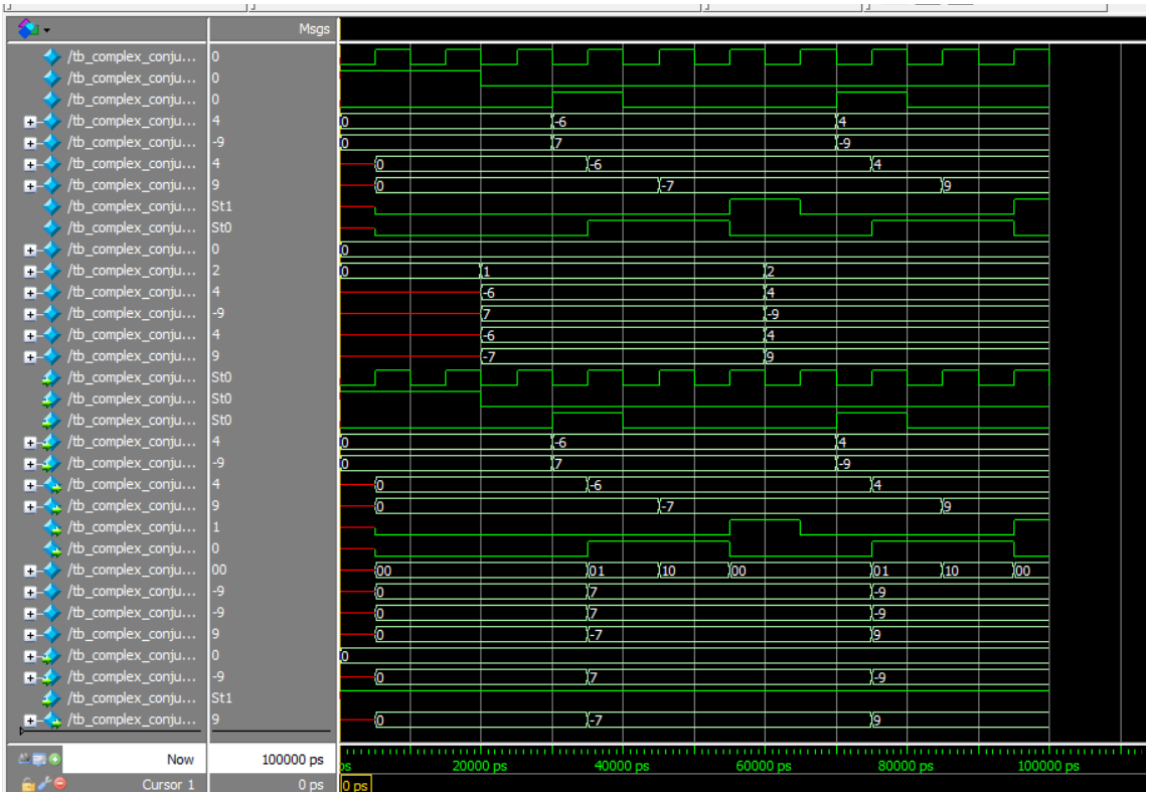
شکل ۵: رونوشت آزمون ضرب مختلط

```
# run -all
# ============================================================
#      Lab 8 - Complex Multiplier Testbench
# ============================================================
# ------------------------------------------------------------
# TEST 1 : COMPLEX MULTIPLICATION
# A = (2,3)   B = (-4,5)
# expected = (-23,-2)
# got      = (-23,-2)
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 2 : COMPLEX MULTIPLICATION
# A = (-10,5)   B = (3,-8)
# expected = (10,95)
# got      = (10,95)
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 3 : COMPLEX MULTIPLICATION
# A = (0,2)   B = (0,2)
# expected = (-4,0)
# got      = (-4,0)
# RESULT   : PASS
# ============================================================
# MULTIPLIER SUMMARY : ALL 3 TESTS PASSED
# ============================================================
# ** Note: $finish    : ../tb/tb_complex_multiplier_mc.v(80)
#    Time: 290 ns  Iteration: 1  Instance: /tb_complex_multiplier_mc
```

شکل ۶: ادامه‌ی رونوشت و گزارش قبولی سه وارسی

اینجا اثر اشتراک منابع را می‌توان مستقیماً در شکل موج دید: میان پالس شروع و پرچم اعتبار، شش چرخهٔ کاری فاصله است — چهار چرخه برای چهار حاصل‌ضرب جزئی و دو چرخه برای دو عمل جمع و تفریق پایانی. اگر طرح موازی بود، همین کار در یک چرخه انجام می‌شد امّا به چهار ضرب‌کننده نیاز داشت. نخستین وارسی، ضرب $(2+j3)(-4+j5)$ را می‌سنجد که نتیجهٔ درست آن $-23-j2$ است.

ج) پیمانه‌های مزدوج و مربع اندازه



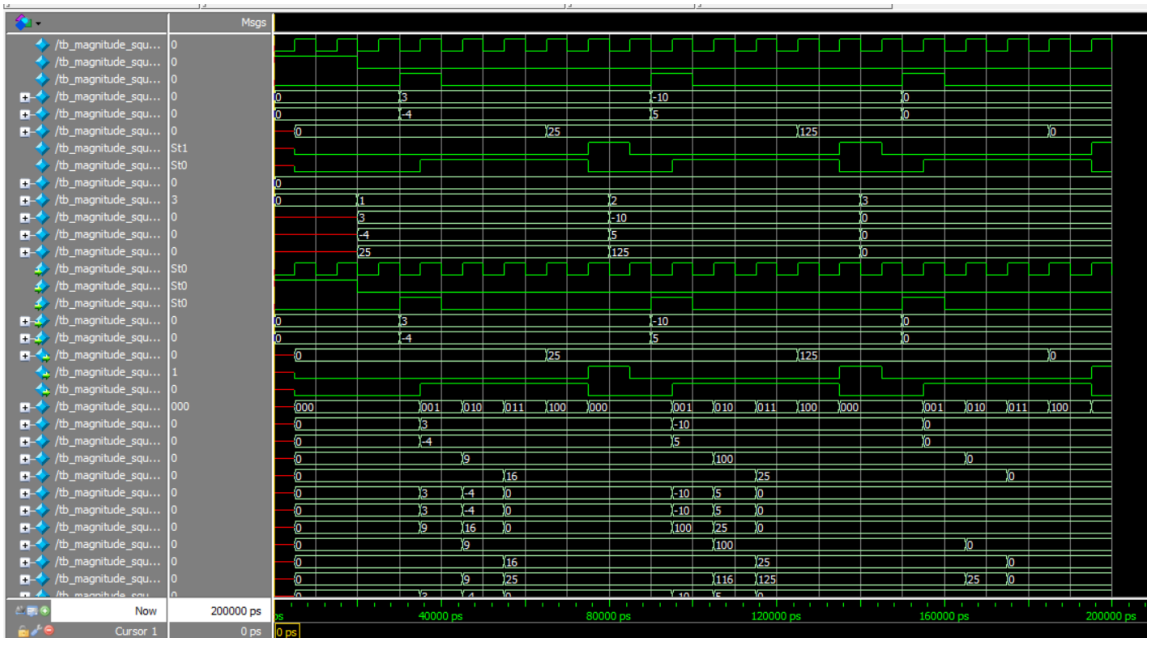
شکل ۷: شکل موج پیمانه‌ی مزدوج

```
# ============================================================
#      Lab 8 - Complex Conjugate Testbench
# ============================================================
# ------------------------------------------------------------
# TEST 1 : CONJUGATE
# input    = (-6,7)
# expected = (-6,-7)
# got      = (-6,-7)
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 2 : CONJUGATE
# input    = (4,-9)
# expected = (4,9)
# got      = (4,9)
# RESULT   : PASS
# ============================================================
# CONJUGATE SUMMARY : ALL 2 TESTS PASSED
# ============================================================
# ** Note: $finish    : ../tb/tb_complex_conjugate.v(74)
#    Time: 100 ns  Iteration: 1  Instance: /tb_complex_conjugate
```

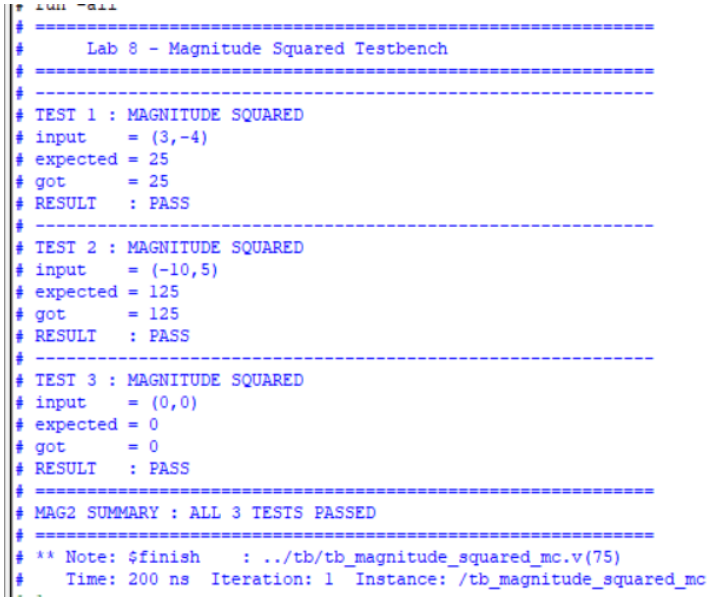
شکل ۸: رونوشت آزمون مزدوج

```
VSIM(paused)> do run_conjugate.do
# Lab 8 - conjugate module test
# transcript on
# if {[file exists work]} { vdel -lib work -all }
# vlib work
# vmap work work
# Modifying modelsim.ini
# vlog ../rtl/shared_arith.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module shared_addsub
# -- Compiling module shared_multiplier
#
# Top level modules:
#       shared_addsub
#       shared_multiplier
# vlog ../rtl/complex_conjugate.v
```

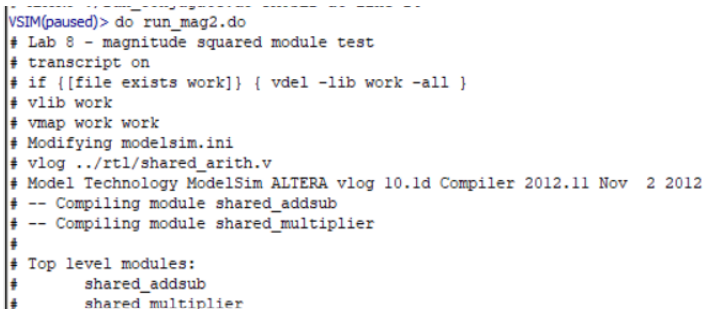
شکل ۹: گزارش قبولی دو وارسی مزدوج



شکل ۱۰: شکل موج پیمانه‌ی مربع اندازه



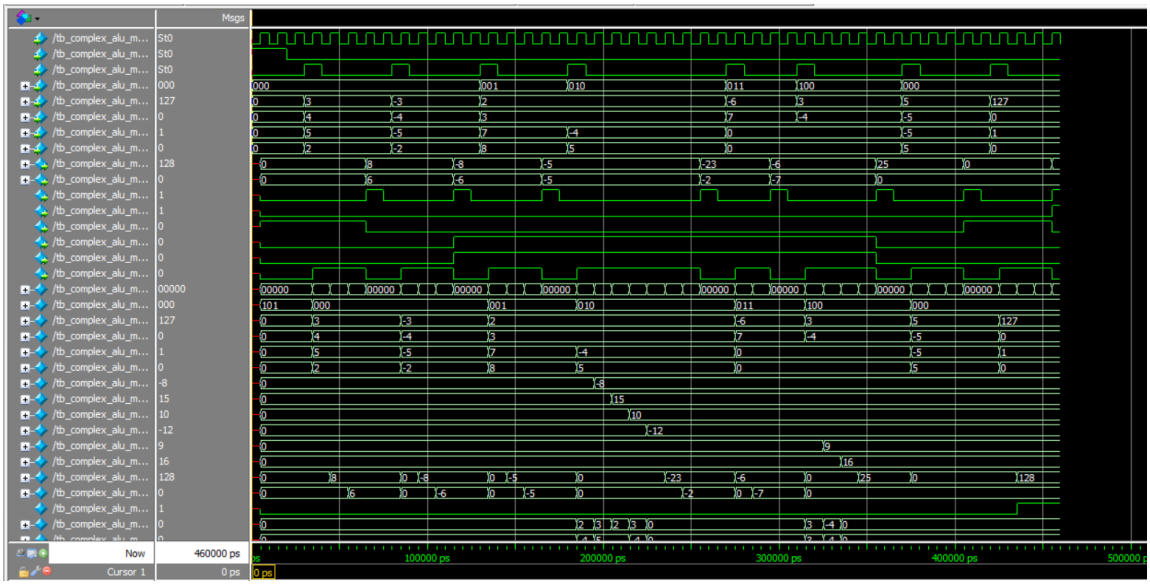
شکل ۱۱: رونوشت آزمون مربع اندازه



شکل ۱۲: گزارش قبولی سه وارسی مربع اندازه

در آزمون مربع اندازه، وارسی نخست ورودی $3 - j4$ را می‌دهد که مربع اندازهٔ آن 25 است؛ توجّه شود که نتیجه با وجود منفی بودن بخش موهومی ورودی، مثبت است، چون هر دو مؤلّفه پیش از جمع مربع می‌شوند. وارسی سوم نیز ورودی صفر را می‌آزماید که حالت مرزی طبیعی این عملیات است.

د) واحد حساب یکپارچه



شکل ۱۳: شکل موج واحد حساب و منطق مختلط

```
VSIM(paused)> do run_alu.do
# Lab 8 - final ALU test
# transcript on
# if {[file exists work]} { vdel -lib work -all }
# vlib work
# vmap work work
# Modifying modelsim.ini
#
# vlog ../rtl/shared_arith.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module shared_addsub
# -- Compiling module shared_multiplier
#
# Top level modules:
#       shared_addsub
#       shared_multiplier
```

شکل ۱۴: رونوشت آزمون واحد حساب

```
# ============================================================
#         Lab 8 - Final Complex ALU Testbench
# ============================================================
# opcode: 000=CADD 001=CSUB 010=CMUL 011=CONJ 100=MAG2
# ------------------------------------------------------------
# TEST 1 : ALU opcode=000
# A=(3,4)  B=(5,2)
# expected result=(8,6) flags O/Z/NR/NI=0000
# got      result=(8,6) flags O/Z/NR/NI=0000
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 2 : ALU opcode=000
# A=(-3,-4)  B=(-5,-2)
# expected result=(-8,-6) flags O/Z/NR/NI=0011
# got      result=(-8,-6) flags O/Z/NR/NI=0011
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 3 : ALU opcode=001
# A=(2,3)  B=(7,8)
# expected result=(-5,-5) flags O/Z/NR/NI=0011
# got      result=(-5,-5) flags O/Z/NR/NI=0011
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 4 : ALU opcode=010
# A=(2,3)  B=(-4,5)
# expected result=(-23,-2) flags O/Z/NR/NI=0011
# got      result=(-23,-2) flags O/Z/NR/NI=0011
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 5 : ALU opcode=011
# A=(-6,7)  B=(0,0)
# expected result=(-6,-7) flags O/Z/NR/NI=0011
# got      result=(-6,-7) flags O/Z/NR/NI=0011
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 6 : ALU opcode=100
# A=(3,-4)  B=(0,0)
# expected result=(25,0) flags O/Z/NR/NI=0000
# got      result=(25,0) flags O/Z/NR/NI=0000
# RESULT   : PASS
# ------------------------------------------------------------
```

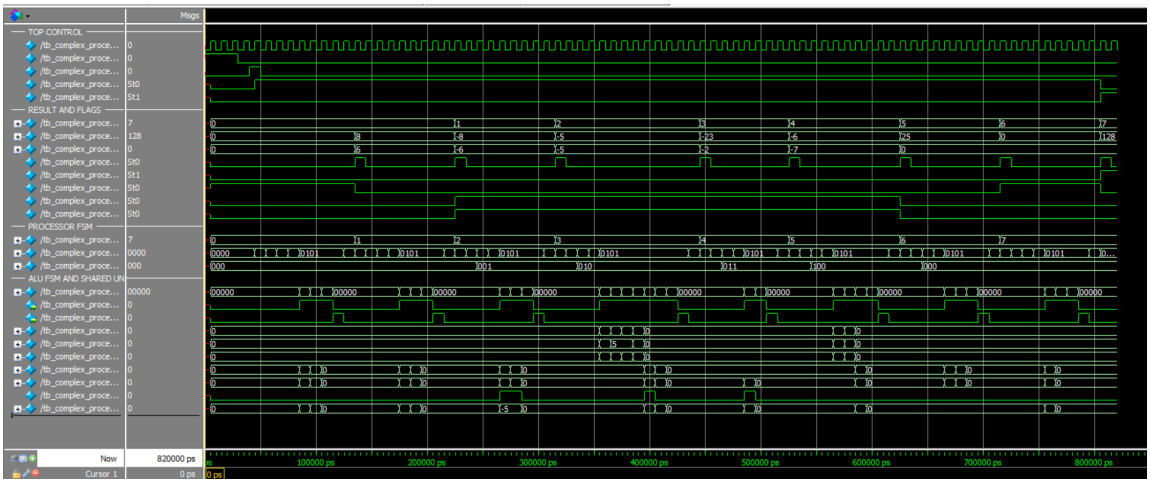
شکل ۱۵: ادامه‌ی رونوشت واحد حساب

```
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 6 : ALU opcode=100
# A=(3,-4)  B=(0,0)
# expected result=(25,0) flags O/Z/NR/NI=0000
# got      result=(25,0) flags O/Z/NR/NI=0000
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 7 : ALU opcode=000
# A=(5,-5)  B=(-5,5)
# expected result=(0,0) flags O/Z/NR/NI=0100
# got      result=(0,0) flags O/Z/NR/NI=0100
# RESULT   : PASS
# ------------------------------------------------------------
# TEST 8 : ALU opcode=000
# A=(127,0)  B=(1,0)
# expected result=(128,0) flags O/Z/NR/NI=1000
# got      result=(128,0) flags O/Z/NR/NI=1000
# RESULT   : PASS
# ============================================================
# ALU SUMMARY : ALL 8 TESTS PASSED
# ============================================================
# ** Note: $finish    : ../tb/tb_complex_alu_mc.v(109)
#    Time: 460 ns  Iteration: 1  Instance: /tb_complex_alu_mc
```

شکل ۱۶: گزارش قبولی هشت وارسی واحد حساب

هشت وارسی این آزمون، هر پنج عملیات را به‌همراه دو حالت پرچمی می‌پوشانند. در شکل موج می‌توان دید که طول اجرای هر عملیات متفاوت است: CONJ کوتاه‌ترین و CMUL بلندترین است. همین تفاوت طول، دلیل وجود حالت انتظار در واحد کنترل است.

۵) رایانه‌ی کامل



شکل ۱۷: شکل موج اجرای کامل برنامه‌ی حافظه

این شکل موج، گویاترین نمای کلّ آزمایش است. ردیف شمارندهٔ نتیجه به‌ترتیب مقدارهای صفر تا هفت را می‌گیرد و در کنار هر یک، بخش حقیقی و موهومی نتیجه دیده می‌شود. با دنبال کردن این ردیف‌ها می‌توان کلّ برنامه را از روی شکل موج بازخواند.

شمارنده	دستور	عملوندها	نتیجه	پرچم‌ها
0	CADD	$(3+j4)$ ،$(5+j2)$	$8+j6$	——
1	CADD	$(-3-j4)$ ،$(-5-j2)$	$-8-j6$	هر دو بخش منفی
2	CSUB	$(2+j3)$ ،$(7+j8)$	$-5-j5$	هر دو بخش منفی
3	CMUL	$(2+j3)$ ،$(-4+j5)$	$-23-j2$	هر دو بخش منفی
4	CONJ	$(-6+j7)$	$-6-j7$	هر دو بخش منفی
5	MAG2	$(3-j4)$	25	——
6	CADD	$(5-j5)$ ،$(-5+j5)$	0	صفر
7	CADD	$(127+j0)$ ،$(1+j0)$	128	سرریز

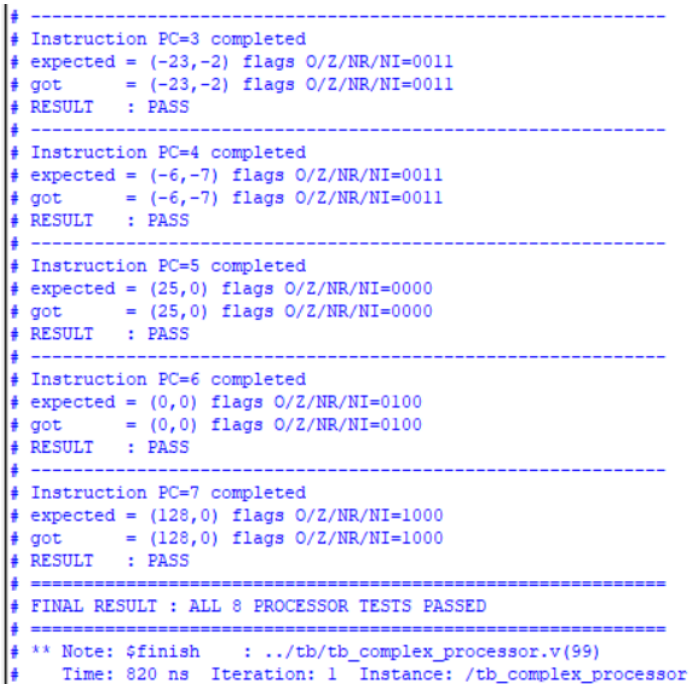
جدول ۳: برنامه‌ی هشت‌دستوری حافظه و نتیجه‌ی هر دستور

```
/SIM(paused)> do run_all.do
# Lab 8 - final processor simulation
# Run this file from the modelsim folder with: do run_all.do
#
# transcript on
# if {[file exists work]} { vdel -lib work -all }
# vlib work
# vmap work work
# Modifying modelsim.ini
#
# compile RTL
# vlog ../rtl/shared_arith.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module shared_addsub
# -- Compiling module shared_multiplier
#
```

شکل ۱۸: آغاز رونوشت اجرای رایانه‌ی کامل

```
# ============================================================
#       Lab 8 - Complete Processor Testbench
# ============================================================
# The memory contains 8 test instructions.
# ------------------------------------------------------------
# Instruction PC=0 completed
# expected = (8,6) flags O/Z/NR/NI=0000
# got      = (8,6) flags O/Z/NR/NI=0000
# RESULT   : PASS
# ------------------------------------------------------------
# Instruction PC=1 completed
# expected = (-8,-6) flags O/Z/NR/NI=0011
# got      = (-8,-6) flags O/Z/NR/NI=0011
# RESULT   : PASS
# ------------------------------------------------------------
# Instruction PC=2 completed
# expected = (-5,-5) flags O/Z/NR/NI=0011
# got      = (-5,-5) flags O/Z/NR/NI=0011
# RESULT   : PASS
# ------------------------------------------------------------
# Instruction PC=3 completed
# expected = (-23,-2) flags O/Z/NR/NI=0011
# got      = (-23,-2) flags O/Z/NR/NI=0011
# RESULT   : PASS
# ------------------------------------------------------------
# Instruction PC=4 completed
# expected = (-6,-7) flags O/Z/NR/NI=0011
# got      = (-6,-7) flags O/Z/NR/NI=0011
# RESULT   : PASS
# ------------------------------------------------------------
# Instruction PC=5 completed
# expected = (25,0) flags O/Z/NR/NI=0000
# got      = (25,0) flags O/Z/NR/NI=0000
# RESULT   : PASS
# ------------------------------------------------------------
# Instruction PC=6 completed
# expected = (0,0) flags O/Z/NR/NI=0100
# got      = (0,0) flags O/Z/NR/NI=0100
# RESULT   : PASS
# ------------------------------------------------------------
# Instruction PC=7 completed
# expected = (128,0) flags O/Z/NR/NI=1000
# got      = (128,0) flags O/Z/NR/NI=1000
# RESULT   : PASS
```

شکل ۱۹: وارسی نتیجه‌ی هشت دستور برنامه



شکل ۲۰: گزارش پایانی و قبولی هر هشت دستور

دو دستور پایانی عمداً برای آزمودن پرچم‌ها انتخاب شده‌اند. دستور ششم دو عدد قرینه را جمع می‌کند تا نتیجه دقیقاً صفر شود و پرچم صفر بالا برود؛ دستور هفتم نیز جمعی می‌سازد که از بازهٔ هشت‌بیتی علامت‌دار بیرون می‌زند تا پرچم سرریز آزموده شود. کلّ برنامهٔ هشت‌دستوری در ۸۲۰ نانوثانیه، یعنی ۸۲ پالس ساعت، به پایان می‌رسد.

۹.۸ سنتز و تحلیل زمانی

الف) مصرف منابع

طرح در Quartus II 13.0.1 و برای خانوادهٔ Cyclone IV GX همگردانی شد. انتخاب قطعه روی حالت خودکار بود و ابزار قطعهٔ EP4CGX22CF19C6 را برگزید. نتیجه ۵۲۰ عنصر منطقی شامل ۴۹۷ تابع ترکیبی و ۲۹۸ ثبات اختصاصی، و ۴۹ پایه است.

Flow Summary	
Flow Status	Successful - Tue Aug 18 14:19:26 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	complex_processor
Top-level Entity Name	complex_processor
Family	Cyclone IV GX
Total logic elements	520 / 21,280 (2 %)
Total combinational functions	497 / 21,280 (2 %)
Dedicated logic registers	298 / 21,280 (1 %)
Total registers	298
Total pins	49 / 167 (29 %)
Total virtual pins	0
Total memory bits	0 / 774,144 (0 %)
Embedded Multiplier 9-bit elements	0 / 80 (0 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Device	EP4CGX22CF19C6
Timing Models	Final

شکل ۲۱: خلاصه‌ی منابع مصرفی رایانه‌ی اعداد مختلط

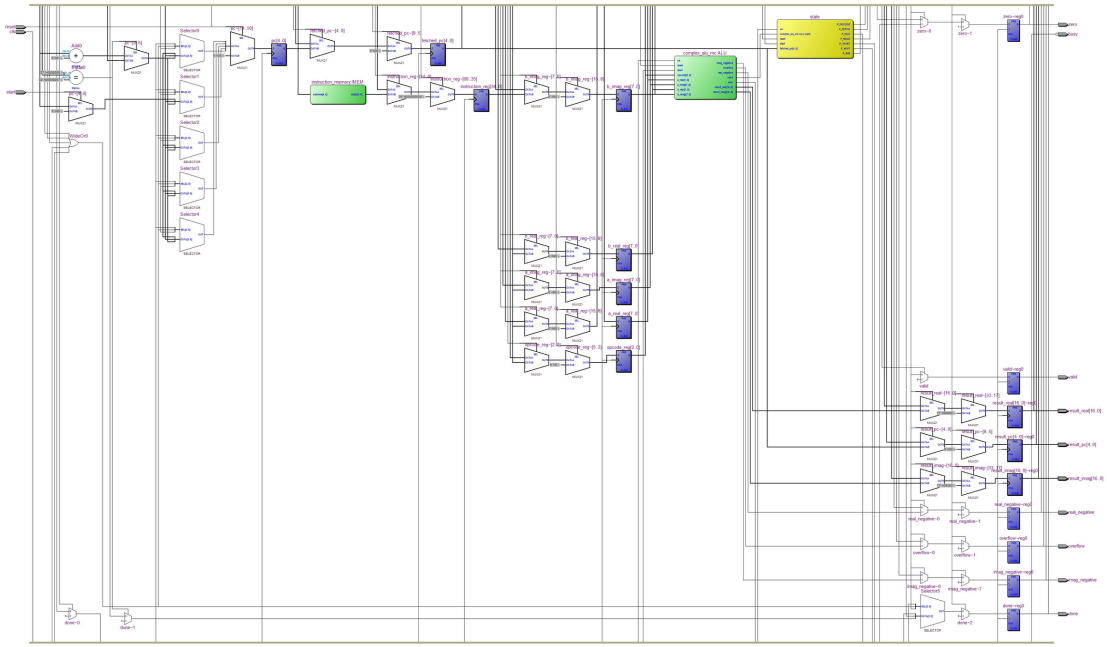
یک عدد این گزارش شایان توجّه ویژه است: شمار ضرب‌کننده‌های نه‌بیتی توکار[11] مصرف‌شده صفر است. یعنی سنتزکننده هیچ بلوک ضرب آمادهٔ تراشه را به کار نگرفته و تنها ضرب‌کنندهٔ طرح را با جدول‌های جست‌وجو ساخته است. این نکته از دو جهت اهمّیّت دارد: نخست آنکه تأیید می‌کند در کلّ طرح واقعاً بیش از یک ضرب‌کننده وجود ندارد، و دوم آنکه بخش بزرگی از ۴۹۷ تابع ترکیبی گزارش‌شده صرف همین یک ضرب‌کنندهٔ هشت در هشت شده است.

شمار ۴۹ پایه نیز از واسط طرح قابل بازبینی است: ساعت، بازنشانی و شروع در ورودی، و دو نتیجهٔ هفده‌بیتی، شمارندهٔ پنج‌بیتی و شش پرچم در خروجی.

ب) نمای سطح انتقال ثبات‌ها و ماشین حالت

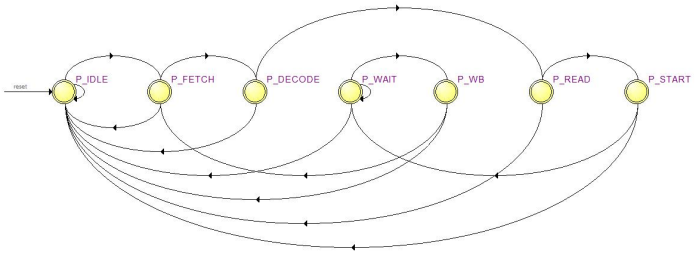
نمای سطح انتقال ثبات‌ها ساختار مورد انتظار را نشان می‌دهد: بلوک حافظهٔ دستور در سمت چپ، ثبات‌های میان‌مرحله‌ای که دستور و عملوندهای رمزگشایی‌شده را نگه می‌دارند، بلوک واحد حساب در میانه، و ثبات‌های خروجی نتیجه و پرچم‌ها در سمت راست.

[11] Embedded Multiplier



شکل ۲۲: نمای سطح انتقال ثبات‌های رایانه‌ی اعداد مختلط

نمای ماشین حالت[۱۲] نیز هفت حالت واحد کنترل و گذارهای میان آن‌ها را نشان می‌دهد. حلقهٔ بازگشتی روی حالت انتظار، همان انتظار برای پایان عملیات واحد حساب است و حلقهٔ بزرگ‌تری که از بازنویسی به واکشی بازمی‌گردد، پیمایش دستورهای پیاپی برنامه را نشان می‌دهد.

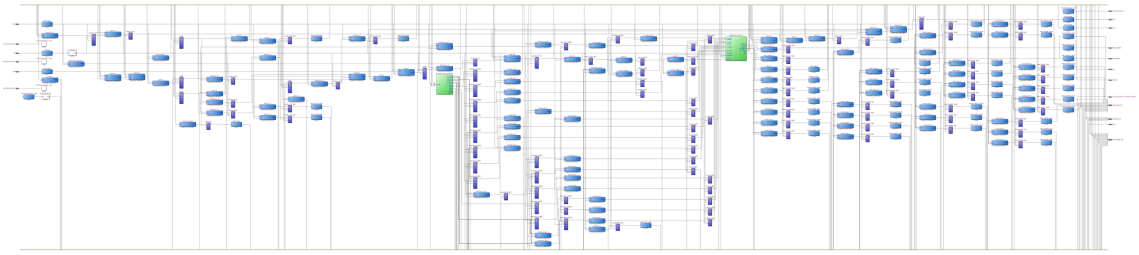


شکل ۲۳: نمای ماشین حالت واحد کنترل

[12]State Machine Viewer



شکل ۲۴: نمای Technology Map Viewer (Post-Mapping)



شکل ۲۵: نمای Technology Map Viewer (Post-Fitting)

ج) تحلیل زمانی

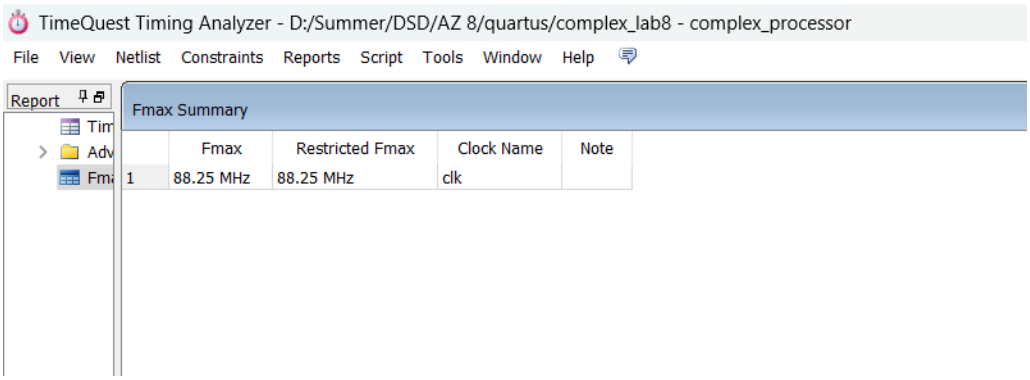
برخلاف چند آزمایش پیشین، برای این طرح یک پروندهٔ قیود طراحی[۱۳] نوشته شد که ساعتی با دورهٔ ۲۰ نانوثانیه، معادل ۵۰ مگاهرتز، تعریف می‌کند و عدم‌قطعیّت ساعت را نیز استخراج می‌نماید.

```
# Lab 8 timing constraint
# 50 MHz clock => 20 ns period
create_clock -name clk -period 20.000 [get_ports {clk}]
derive_clock_uncertainty
```

کد ۱۱: پرونده‌ی قیود طراحی

تحلیل‌گر زمانی[۱۴] بیشینه‌بسامد[۱۵] را 88.25 MHz گزارش می‌کند و این بار بیشینه‌بسامد مقیّد نیز همین مقدار است؛ یعنی محدودکنندهٔ سرعت، منطق درونی طرح است و نه نرخ تغییر پایه‌ها.

[13] Synopsys Design Constraints
[14] TimeQuest Timing Analyzer
[15] Maximum Frequency (Fmax)



شکل ۲۶: گزارش بیشینه‌بسامد رایانه‌ی اعداد مختلط

از این عدد می‌توان حاشیهٔ زمانی را بازسازی کرد: بلندترین مسیر داده برابر $1/88.25\,\text{MHz} \approx 11.33$ نانوثانیه است، پس در برابر قید ۲۰ نانوثانیه‌ای حاشیه‌ای نزدیک به 8.7 نانوثانیه باقی می‌ماند و طرح قید خود را با اطمینان برآورده می‌کند. مجموع حاشیه‌های منفی در همهٔ تحلیل‌ها صفر است و حاشیهٔ نگهداری نیز مثبت می‌ماند.

بیشینه‌بسامد این طرح به‌روشنی کمتر از آزمایش‌های پیشین است و دلیل آن هم آشکار است: مسیر بحرانی از ضرب‌کنندهٔ هشت در هشتی می‌گذرد که با جدول‌های جست‌وجو ساخته شده و پس از آن به جمع‌کنندهٔ هفده‌بیتی می‌رسد. این همان بهایی است که برای اشتراک منابع و پرهیز از ضرب‌کننده‌های موازی پرداخته می‌شود.

با توجّه به برگزاری مجازی آزمایشگاه، ارزیابی این آزمایش بر پایهٔ شبیه‌سازی، سنتز و تحلیل زمانی نرم‌افزاری انجام شده و در پرونده‌های پیکربندی پروژه تخصیص پایه‌ای برای برد ثبت نشده است؛ ازاین‌رو ادّعایی دربارهٔ آزمون فیزیکی روی برد مطرح نمی‌شود.

۱۰.۸ نتایج

- پیمانه‌های جمع و تفریق و ضرب اعداد مختلط، مطابق خواستهٔ دستورکار، جداگانه پیاده‌سازی و هر یک با میز آزمون خودآزمای خود آزموده شدند.
- دو پیمانهٔ مزدوج و مربع اندازه نیز افزوده شدند تا مجموعهٔ عملیات واحد حساب کامل شود.
- قید دستورکار دربارهٔ استفادهٔ یک‌بارهٔ واحدهای حسابی به‌طور کامل رعایت شد: در کلّ طرح تنها یک ضرب‌کننده و یک جمع‌کننده وجود دارد و همهٔ عملیات با اشتراک زمانی از آن‌ها استفاده می‌کنند.
- صفر بودن ضرب‌کننده‌های توکار در گزارش سنتز، این ادّعا را به‌صورت مستقل تأیید می‌کند.
- رایانهٔ خواسته‌شده با حافظهٔ ۳۲ کلمه‌ای ساخته شد و برنامهٔ هشت‌دستوری آن، شامل هر پنج عملیات و دو حالت پرچمی، بدون خطا اجرا شد.

- در مجموع بیست و هشت وارسی در شش میز آزمون انجام شد و همگی با موفّقیّت گذشتند.
- سنتز با ۵۲۰ عنصر منطقی، ۲۹۸ ثبات و ۴۹ پایه انجام شد و تحلیل زمانی با قید ۵۰ مگاهرتزی بیشینه‌بسامد 88.25 MHz و حاشیهٔ مثبت داد.

نتیجه‌گیری

در این آزمایش یک رایانهٔ پایه برای اعداد مختلط از پایین به بالا ساخته شد: نخست دو بلوک حسابی مشترک، سپس چهار پیمانهٔ عملیاتی، آنگاه یک واحد حساب یکپارچه و سرانجام واحد کنترلی که برنامه‌ای را از حافظه می‌خواند و اجرا می‌کند. آنچه شکل این معماری را تعیین کرد، قید صریح دستورکار دربارهٔ استفادهٔ یک‌بارهٔ واحدهای حسابی بود؛ قیدی که طرح را ناگزیر از چندچرخه‌ای شدن کرد و در ازای زمان اجرای بیشتر، مساحت را چشمگیر کاهش داد. همین قید بود که ما را واداشت میان دو خواستهٔ ناسازگار دستورکار —— اجرای پایپ‌لاین و اشتراک کامل منابع —— آگاهانه انتخاب کنیم و دلیل انتخاب خود را در گزارش بیاوریم. سرانجام، گزارش سنتز با صفر اعلام کردن ضرب‌کننده‌های توکار، نشان داد که ادّعای اشتراک منابع تنها ادّعایی در سطح کد نیست و در سخت‌افزار ساخته‌شده نیز برقرار است.